



SOBRECARGA DE OPERADORES

PARTE 2

Prof. Dr. Valter Vieira de Camargo

POO - 2025/1

Comutatividade e Operadores Binários

- Um operador binário é comutativo se a ordem dos operandos não afeta o resultado $\rightarrow A + B = B + A$
- O problema é quando estamos sobrecarregado um operador para fazer operações com **classes diferentes**. Isso exige atenção especial.
- Suponha que queremos somar um objeto da classe Ponto com um int
 - $p1 + 5$
 - $5 + p1$
- Para conseguir isso, normalmente deve-se declarar a função de sobrecarga como função “**não-membro**” ou como friend

Comutatividade e Operadores Binários

- Função de sobrecarga como uma função membro
 - Quando a função de sobrecarga é função membro de uma classe, o operando à esquerda obrigatoriamente deve ser um objeto daquela classe
 - Exemplo:
 - Suponha $\rightarrow$ `Ponto::operator+(int valor)`
 - Isso permite fazer `p1 + 5`
 - Mas não permite fazer `5 + p1`
 - Isso ocorre porque 5 não é um objeto ponto e não pode chamar a função de sobrecarga

Função de sobrecarga global

- Para resolver, uma forma é declarar a função de sobrecarga como uma função global
- Nesse caso, ela deve receber ambos os operandos como parâmetro
 - Exemplo:

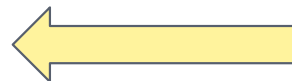
```
operator+(const Ponto& p, int valor) e operator+(int  
valor, const Ponto& p).
```

- Se a função global precisar acessar membros privados da classe, ela pode ser declarada como uma função friend.

```
#include <iostream>
```

```
class Ponto {  
private:  
    int x;  
    int y;  
  
public:  
    Ponto(int coord_x = 0, int coord_y = 0) : x(coord_x), y(coord_y) {}  
  
    void exibir() const {  
        std::cout << "(" << x << ", " << y << " )";  
    }  
  
    // Declarações de operadores + como funções friend globais  
    // Isso permite que essas funções acessem os membros privados x e y  
    friend Ponto operator+(const Ponto& p, int valor);  
    friend Ponto operator+(int valor, const Ponto& p);  
};
```

```
// Implementação da sobrecarga do operador + (Ponto + int) (Primeiro operando é Ponto, segundo é int)  
Ponto operator+(const Ponto& p, int valor) {  
    // Cria um novo Ponto com as coordenadas somadas  
    return Ponto(p.x + valor, p.y + valor);  
}  
  
// Implementação da sobrecarga do operador + (int + Ponto) (Primeiro operando é int, segundo é Ponto)  
Ponto operator+(int valor, const Ponto& p) {  
    // Para manter a comutatividade, a lógica é a mesma ou pode chamar a primeira versão  
    return Ponto(p.x + valor, p.y + valor);  
    // Ou alternativamente, para reutilizar código:  
    // return p + valor; // Chama a versão Ponto + int  
}
```



Interessante !!

```
int main() {
```

```
    Ponto p1(10, 20);  
    int num = 5;
```

```
    // Teste 1: Ponto + int  
    Ponto p2 = p1 + num;  
    std::cout << "p1 + " << num << " = ";  
    p2.exibir();  
    std::cout << std::endl; // Esperado: (15, 25)
```

```
    // Teste 2: int + Ponto (demonstrando a comutatividade)  
    Ponto p3 = num + p1;  
    std::cout << num << " + p1 = ";  
    p3.exibir();  
    std::cout << std::endl; // Esperado: (15, 25)
```

// Outros testes

```
Ponto p4 = Ponto(1, 2) + 10;  
std::cout << "(1, 2) + 10 = ";  
p4.exibir();  
std::cout << std::endl; // Esperado: (11, 12)
```

```
Ponto p5 = 20 + Ponto(3, 4);  
std::cout << "20 + (3, 4) = ";  
p5.exibir();  
std::cout << std::endl; // Esperado: (23, 24)
```

```
    return 0;
```

```
}
```

Dicas

Retorno por Valor: Para operadores binários que criam um novo valor (como **+**, **-**, *****), geralmente é melhor retornar o resultado por valor (**Ponto operator+(...)**).

Constância: Se um operador não modifica o objeto no qual ele é chamado (ou os objetos passados como referência), declare os parâmetros como **const** referências (**const Ponto&**).

Sobrecarga de << e >>

- cin e cout são objetos chamados de “streams” (fluxo de dados)
 - cout << “olá” → envia dados para o fluxo de saída → tela
 - cin >> idade → capta dados do teclado e armazena na variável
- `std::cout` é um objeto da classe `std::ostream`.
- O operador << foi sobrecarregado para a classe `std::ostream` para que ele possa "receber" diferentes tipos de dados (`const char*` para strings, `int` para inteiros, `double` para floats, etc.) e enviá-los para o console

```
std::cout << "Olá, mundo!" << std::endl;  
int idade = 30;  
std::cout << "Sua idade é: " << idade << " anos." << std::endl;
```

Sobrecarga de << e >>

- É bastante comum sobrecarregar esses operadores para permitir imprimir de forma personalizada
- << é usado para enviar dados para uma Stream de saída
- Exemplo

```
int main() {
    Ponto p1(10, 20);
    Ponto p2(5, 7);
    Ponto p3(0, 0);

    // Usando o operador << sobrecarregado para imprimir objetos Ponto
    std::cout << "Ponto 1: " << p1 << std::endl; // Saída: Ponto 1: (10, 20)
    std::cout << "Ponto 2: " << p2 << std::endl; // Saída: Ponto 2: (5, 7)

    // Encadeando operações de saída com o objeto Ponto
    std::cout << "Ponto 1 e Ponto 2: " << p1 << " e " << p2 << std::endl;

    // Imprimindo um ponto construído na hora
    std::cout << "Ponto temporario: " << Ponto(3, 4) << std::endl;
                                     // Saída: Ponto temporario: (3, 4)

    return 0;
}
```



```
#include <iostream> // Necessário para std::cout, std::ostream, std::endl
```

```
class Ponto {
```

```
private:
```

```
    int x;
```

```
    int y;
```

```
public:
```

```
    // Construtor
```

```
    Ponto(int coord_x = 0, int coord_y = 0) : x(coord_x), y(coord_y) {}
```

```
    // Getter para exibir o ponto (apenas para demonstração,
```

```
    // o operador << vai substituir a necessidade desta função para impressão)
```

```
    void exibir() const {
```

```
        std::cout << "(" << x << ", " << y << ")";
```

```
    }
```

```
    // Declara o operador << como uma função friend.
```

```
    // Isso é necessário porque o primeiro operando (std::ostream) não é da classe Ponto,
```

```
    // e a função precisa acessar os membros privados (x, y) de Ponto.
```

```
    // Retorna uma referência a std::ostream para permitir o encadeamento de operações.
```

```
friend std::ostream& operator<<(std::ostream& os, const Ponto& p);
```

```
};
```

// Implementação da sobrecarga do operador << para a classe Ponto
// Recebe uma referência à stream de saída (os) e uma referência constante ao
objeto Ponto (p).

```
std::ostream& operator<<(std::ostream& os, const Ponto& p) {
```

```
    // Aqui, formatamos como "(x, y)".
```

```
    os << "(" << p.x << ", " << p.y << ");
```

```
    // Retorna a referência à stream para que múltiplas operações << possam ser  
    encadeadas.
```

```
    return os;
```

```
}
```

Sobrecarga de >>

- Para ler dados de entrada, frequentemente usamos o formato abaixo com `cin >>`
- `std::cin` é um objeto da classe `std::istream`. O operador `>>` foi sobrecarregado para que ele possa "extrair" dados da entrada padrão e armazená-los em variáveis de diferentes tipos.
- Assim como o `<<`, você pode sobrecarregar o operador `>>` para suas próprias classes para ler dados de forma personalizada:

```
int numero;  
std::cout << "Digite um numero: ";  
std::cin >> numero;  
std::cout << "Voce digitou: " << numero << std::endl;
```

QUEREMOS FAZER ISSO

```
int main() {  
    Ponto meuPonto; // Objeto Ponto para armazenar a entrada do usuário  
    Ponto outroPonto; // Outro objeto para demonstração  
  
    std::cout << "Por favor, digite as coordenadas para 'meuPonto' no formato (X,Y): ";  
    std::cin >> meuPonto; // Usa a sobrecarga do operador >>  
    std::cout << "Voce digitou: " << meuPonto << std::endl; // Usa a sobrecarga do operador <<  
  
    std::cout << "\nAgora, digite as coordenadas para 'outroPonto': ";  
    std::cin >> outroPonto;  
    std::cout << "Voce digitou: " << outroPonto << std::endl;  
  
    // Demonstração de tratamento de erro com entrada inválida  
    std::cout << "\nTestando com entrada invalida. Tente digitar algo como 'abc' ou '10 20': ";  
    Ponto pontoInvalido;  
    std::cin >> pontoInvalido; // Isso provavelmente causará um erro de input  
    std::cout << "Ponto lido (pode estar incorreto devido ao erro): " << pontoInvalido << std::endl;  
  
    return 0;  
}
```

```
#include <iostream> // Para std::cout, std::cin, std::istream, std::ostream, std::endl
#include <limits>    // Para std::numeric_limits, útil no tratamento de erros de input
```

```
class Ponto {
```

```
private:
```

```
    int x;
```

```
    int y;
```

```
public:
```

```
    // Construtor
```

```
    Ponto(int coord_x = 0, int coord_y = 0) : x(coord_x), y(coord_y) {}
```

```
    // Sobrecarga do operador << para saída (já existente)
```

```
    // Permite imprimir Ponto diretamente usando std::cout
```

```
    friend std::ostream& operator<<(std::ostream& os, const Ponto& p) {
```

```
        os << "(" << p.x << ", " << p.y << " )";
```

```
        return os;
```

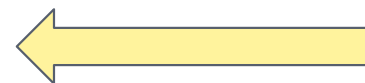
```
    }
```

```
    // Declaração da sobrecarga do operador >> como friend
```

```
    // Permite ler Ponto diretamente usando std::cin
```

```
    // Note que 'p' não é const, pois seus membros serão modificados.
```

```
    friend std::istream& operator>>(std::istream& is, Ponto& p);
```



```
};
```

```
// Implementação da sobrecarga do operador >> para a classe Ponto
// Recebe uma referência à stream de entrada (is) e uma referência ao objeto Ponto (p).
```

```
std::istream& operator>>(std::istream& is, Ponto& p) {
```

```
    char ch; // Variável temporária para consumir os caracteres de formatação (parênteses, vírgula)
```

```
    // Tenta ler os dados no formato esperado: (X,Y)
```

```
    // is >> ch -> tenta ler o '('
```

```
    // >> p.x -> tenta ler o valor de X
```

```
    // >> ch -> tenta ler a ','
```

```
    // >> p.y -> tenta ler o valor de Y
```

```
    // >> ch -> tenta ler o ')'
```

```
    is >> ch >> p.x >> ch >> p.y >> ch;
```

```
    // --- Tratamento de Erros de Entrada ---
```

```
    // Verifica se a leitura falhou (ex: usuário digitou texto em vez de número)
```

```
    // ou se o último caractere lido não foi o ')' esperado.
```

```
    if (is.fail() || ch != ')') {
```

```
        // Limpa o estado de erro da stream (sem isso, futuras leituras falhariam)
```

```
        is.clear();
```

```
        // Ignora o restante da linha de entrada para limpar o buffer
```

```
        // std::numeric_limits<std::streamsize>::max() é o maior número de caracteres para ignorar
```

```
        // '\n' é o delimitador que fará ele parar de ignorar
```

```
        is.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

```
        // Imprime uma mensagem de erro para o usuário
```

```
        std::cerr << "Erro de formato! Por favor, digite as coordenadas no formato (X,Y).\n";
```

```
    }
```

fail() é falha de leitura. Por exemplo, esperando um inteiro e o usuário digitou um caracter alfanumerico.

estamos dizendo para o **ignore()**: "ignore até o final do buffer, não importa o quão grande ele seja." É uma forma de dizer "ignore um número arbitrariamente grande de caracteres".

Observe que a validação não está completa ...
E se eu digitar
)10,20
A10B20)

```
// Implementação da sobrecarga do operador >> para a classe Ponto
// Recebe uma referência à stream de entrada (is) e uma referência ao objeto Ponto (p).
```

```
std::istream& operator>>(std::istream& is, Ponto& p) {
```

```
    char ch; // Variável temporária para consumir os caracteres de formatação (parênteses, vírgula)
```

```
    // Tenta ler os dados no formato esperado: (X,Y)
```

```
    // is >> ch -> tenta ler o '('
```

```
    // >> p.x -> tenta ler o valor de X
```

```
    // >> ch -> tenta ler a ','
```

```
    // >> p.y -> tenta ler o valor de Y
```

```
    // >> ch -> tenta ler o ')'
```

```
    is >> ch >> p.x >> ch >> p.y >> ch;
```

```
    // --- Tratamento de Erros de Entrada ---
```

```
    // Verifica se a leitura falhou (ex: usuário digitou texto em vez de número)
```

```
    // ou se o último caractere lido não foi o ')' esperado.
```

```
    if (is.fail() || ch != ')') {
```

```
        // Limpa o estado de erro da stream (sem isso, futuras leituras falhariam)
```

```
        is.clear();
```

```
        // Ignora o restante da linha de entrada para limpar o buffer
```

```
        // std::numeric_limits<std::streamsize>::max() é o maior número de caracteres para ignorar
```

```
        // '\n' é o delimitador que fará ele parar de ignorar
```

```
        is.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

```
        // Imprime uma mensagem de erro para o usuário
```

```
        std::cerr << "Erro de formato! Por favor, digite as coordenadas no formato (X,Y).\n";
```

```
    }
```

estamos dizendo para o `ignore()`: "ignore *até* o final do buffer, não importa o quão grande ele seja." É uma forma de dizer "ignore um número arbitrariamente grande de caracteres".

Observe que a validação não está completa ...

E se eu digitar isso abaixo?

)10,20)

A10B20)

```
// Implementação da sobrecarga do operador >> para a classe Ponto
// Recebe uma referência à stream de entrada (is) e uma referência ao objeto Ponto (p).
```

```
std::istream& operator>>(std::istream& is, Ponto& p) {
```

```
    char ch; // Variável temporária para consumir os caracteres de formatação (parênteses, vírgula)
```

```
    // Tenta ler os dados no formato esperado: (X,Y)
```

```
    // is >> ch -> tenta ler o '('
```

```
    // >> p.x -> tenta ler o valor de X
```

```
    // >> ch -> tenta ler a ','
```

```
    // >> p.y -> tenta ler o valor de Y
```

```
    // >> ch -> tenta ler o ')'
```

```
    is >> ch >> p.x >> ch >> p.y >> ch;
```

```
    // --- Tratamento de Erros de Entrada ---
```

```
    // Verifica se a leitura falhou (ex: usuário digitou texto em vez de número)
```

```
    // ou se o último caractere lido não foi o ')' esperado.
```

```
    if (is.fail() || ch != ')') {
```

```
        // Limpa o estado de erro da stream (sem isso, futuras leituras falhariam)
```

```
        is.clear();
```

```
        // Ignora o restante da linha de entrada para limpar o buffer
```

```
        // std::numeric_limits<std::streamsize>::max() é o maior número de caracteres para ignorar
```

```
        // '\n' é o delimitador que fará ele parar de ignorar
```

```
        is.ignore(
```

```
        // In
```

```
        std
```

```
    }
```

estamos dizendo para o `ignore()`: "ignore *até* o final do buffer, não importa o quão grande ele seja." É uma forma de dizer "ignore um número arbitrariamente grande de caracteres".

Refaça este exemplo fazendo uma validação completa na entrada esperada. O usuário digita a entrada no formato esperado. Se algo estiver errado, ele recebe uma mensagem de erro e deve ter outra chance de digitar corretamente.

EXERCÍCIO SOBBRECARGA PARA ARRAYS

□ Arrays em C++

- Não há verificação de limites
- Não se compara com `==`
- Não há atribuição
- Não se pode ler/escrever arrays inteiros de uma única vez
 - Um elemento por vez

□ Exemplo: Implemente uma Classe **Array** com as seguintes características

- Verificação de limites
- Atribuição
- Arrays que conheçam o próprio tamanho
- Entrada e saída com `<<` e `>>`
- Comparações com `==` e `!=`

EXERCÍCIO SOBBRECARGA PARA ARRAYS

ALGUMAS DICAS ...

- Crie um array de ponteiros para inteiros
- use sempre o `size_t` para índices, pois é mais seguro e não permite negativo
- Não esqueça de criar o destrutor, pois trata-se de uma classe que tem ponteiros, então, vocês é que devem fazer o gerenciamento de memória
- Crie também um construtor de cópia para garantir um “deep copy”
- no operador `=`, cuidar para que não seja possível atribuir o mesmo array a ele mesmo
- No operador de `=` trate o fato de atribuir vetores com tamanhos diferentes.
- Analisem a função “copy” da biblioteca `std`, pois ela facilita a cópia de vetores

COMO DEVE SER O MAIN ?

```
int main() {
    std::cout << "--- Teste da Classe Array ---" << std::endl;

    // 1. Array que conhece o próprio tamanho e inicialização
    Array meuArray(5); // Cria um array de 5 elementos, inicializados com 0
    std::cout << "Meu Array (tamanho " << meuArray.obterTamanho() << "): " << meuArray << std::endl;

    // 2. Verificação de limites e acesso a elementos
    try {
        meuArray[0] = 10;
        meuArray[4] = 50;
        // meuArray[5] = 100; // Isso lançaria uma exceção!
        std::cout << "Apos atribuicao de elementos: " << meuArray << std::endl;

        std::cout << "Elemento na posicao 2: " << meuArray[2] << std::endl;
        // Teste de acesso fora do limite (descomente para ver a exceção)
        // std::cout << "Tentando acessar indice 5: " << meuArray[5] << std::endl;
    } catch (const std::out_of_range& e) {
        std::cerr << "ERRO: " << e.what() << std::endl;
    }

    // 3. Atribuição de Arrays
    Array outroArray(5);
    outroArray = meuArray; // Atribuição de um Array para outro
    std::cout << "Outro Array (copia de Meu Array): " << outroArray << std::endl;

    Array umNovoArray(3);
    umNovoArray = outroArray; // Atribuição com tamanhos diferentes
    std::cout << "Um Novo Array (copia de Outro Array, tamanho alterado): " << umNovoArray << std::endl;

    // 4. Entrada de dados com >>
    Array arrayLido(2);
    std::cout << "\n--- Teste de Entrada de Dados ---" << std::endl;
```

COMO DEVE SER O MAIN ?

// 5. Comparações com == e !=

```
std::cout << "\n--- Teste de Comparacao ---" << std::endl;
```

```
Array a1(3), a2(3), a3(3);
```

```
a1[0] = 1; a1[1] = 2; a1[2] = 3;
```

```
a2[0] = 1; a2[1] = 2; a2[2] = 3;
```

```
a3[0] = 4; a3[1] = 5; a3[2] = 6;
```

```
std::cout << "a1: " << a1 << std::endl;
```

```
std::cout << "a2: " << a2 << std::endl;
```

```
std::cout << "a3: " << a3 << std::endl;
```

```
if (a1 == a2) {
```

```
    std::cout << "a1 e a2 sao iguais." << std::endl;
```

```
} else {
```

```
    std::cout << "a1 e a2 sao diferentes." << std::endl;
```

```
}
```

```
if (a1 != a3) {
```

```
    std::cout << "a1 e a3 sao diferentes." << std::endl;
```

```
} else {
```

```
    std::cout << "a1 e a3 sao iguais." << std::endl;
```

```
}
```

```
Array a4(2); // Diferente tamanho
```

```
if (a1 == a4) {
```

```
    std::cout << "a1 e a4 sao iguais (erro de logica)." << std::endl;
```

```
} else {
```

```
    std::cout << "a1 e a4 sao diferentes (correto)." << std::endl;
```

```
}
```

```
std::cout << "\n--- Fim do Teste ---" << std::endl;
```

```
return 0;
```